



The **psql** Cheatsheet

A friendly, visual guide to PostgreSQL's interactive terminal

PostgreSQL 16+

Beginner → Pro

Backslash · SQL · Tricks

2026 Edition

1 Get Connected

⚡ Anatomy of a connection

USER
-U alice

HOST
-h db.io

PORT
-p 5432

DATABASE
-d shop

```
# classic flag form
psql -h db.io -p 5432 -U alice -d shop

# URI form (great for scripts & copy-paste)
psql "postgresql://alice:secret@db.io:5432/shop?sslmode=require"

# env-var form (no secrets in shell history)
PGPASSWORD=secret psql -U alice -d shop
```

Pro tip: store credentials in ~/.pgpass with `chmod 600` and skip the password prompt forever.

🔄 Switch / reconnect inside psql

<code>\c db</code>	connect to another database
<code>\c db user</code>	connect as another user
<code>\conninfo</code>	show current connection details
<code>\password</code>	set password securely (no echo)
<code>\q</code> · <code>Ctrl + D</code>	quit psql

? Where am I?

<code>SELECT current_database();</code>	db name
<code>SELECT current_user;</code>	login role
<code>SELECT current_schema();</code>	active schema
<code>SELECT version();</code>	server version
<code>SHOW search_path;</code>	name resolution order

2 Explore the Schema

i Anatomy of a backslash command

`\d+ public.users`

● backslash starts a meta-command

● verb · what to do (d = describe)

● modifier · + verbose, S system

● argument · object pattern (* ?)

Describe objects

<code>\d</code>	list tables, views, sequences
<code>\d name</code>	describe one object
<code>\d+ name</code>	verbose: storage, stats
<code>\dt</code>	tables only
<code>\dv</code>	views
<code>\di</code>	indexes
<code>\ds</code>	sequences
<code>\dm</code>	materialized views
<code>\df</code>	functions
<code>\dn</code>	schemas (namespaces)
<code>\du</code>	users / roles
<code>\dx</code>	installed extensions
<code>\dp</code>	privileges per object

Patterns: `\dt public.*` · append `S` for system (`\dtS`).

See what's inside

```
\dt+           -- tables w/ size
\d users       -- columns + indexes + FKs
\d+ users      -- + storage, comments,
stats
\sf get_user(int) -- show function source
\sv active_users -- show view definition
\ef get_user(int) -- edit function in
$EDITOR
```

Fun fact: `\d` is short for *describe*. Add letters to narrow scope, modifiers to widen it.

3 Run Queries Like a Pro

The query buffer

<code>;</code>	execute current statement
<code>\g [file]</code>	re-run last query (or save)
<code>\gx</code>	run + force expanded
<code>\gexec</code>	run, then exec each row as SQL
<code>\watch n</code>	re-run every <code>n</code> seconds
<code>\e</code>	edit last query in <code>\$EDITOR</code>
<code>\p · \r · \w</code>	print · reset · write buffer
<code>\i file.sql</code>	run commands from a file
<code>\ir file.sql</code>	include relative to script

Magic of `\gexec`

```
-- Drop every table starting with "tmp_"
SELECT 'DROP TABLE ' || tablename || ';'
FROM pg_tables
WHERE tablename LIKE 'tmp\_%'
\gexec
```

Heads up: `\gexec` runs whatever your query returned. Test with `SELECT` first, then swap in the action.

Keyboard shortcuts (readline)

<code>Ctrl + R</code>	reverse-search history	<code>Ctrl + A</code>	jump to start of line	<code>Ctrl + L</code>	clear screen
<code>↑</code> <code>↓</code>	walk through history	<code>Ctrl + E</code>	jump to end of line	<code>Ctrl + C</code>	cancel current input
<code>Tab</code>	autocomplete tables, cols, kw	<code>Ctrl + U</code>	clear line before cursor	<code>Ctrl + D</code>	quit on empty line

4 Make Output Beautiful

Output formats at a glance

Aligned (default)

```
id | name | age
---+-----+-----
1  | alice | 30
2  | bob   | 27
```

Expanded · \x

```
-[ RECORD 1 ]-
id   | 1
name | alice
age  | 30
```

CSV · --csv

```
id,name,age
1,alice,30
2,bob,27
```

Formatting toggles

<code>\x [on off auto]</code>	expanded display
<code>\pset format <i>fmt</i></code>	aligned · csv · html · latex
<code>\pset border 2</code>	draw full borders
<code>\pset null 'Ø'</code>	placeholder for NULLs
<code>\pset pager off</code>	disable pager (good in scripts)
<code>\pset linestyle unicode</code>	pretty box-drawing borders
<code>\pset numericlocale on</code>	thousand separators
<code>\timing on</code>	show duration of every query

Send results elsewhere

<code>\o <i>file.txt</i></code>	redirect output to file
<code>\o <i>less</i></code>	pipe to a shell command
<code>\o</code>	stop redirecting (back to term)
<code>\g <i>file.csv</i></code>	save just that result
<code>\H</code>	toggle HTML output mode
<code>\copy ... TO ...</code>	client-side CSV export (\$6)

Tip: `\timing on + \watch 2` = a free dashboard.

5 Variables & Scripting

Set & use variables

```
\set uid 42
\set tbl 'users'

-- numeric / identifier substitution
SELECT * FROM :tbl WHERE id = :uid;

-- quote a value safely
SELECT * FROM users WHERE name = :'name';

-- quote an identifier safely
SELECT * FROM : "tbl";
```

<code>\set</code>	list all variables
<code>\unset <i>name</i></code>	remove variable
<code>\echo :var</code>	print value (debugging)

Conditional & safer scripts

```
\set ON_ERROR_STOP on -- abort on first error
\set AUTOCOMMIT off

\if :{?some_var}
    SELECT 'var is set';
\else
    \echo 'please pass -v some_var=...'
\endif
```

Recipe: launch with `psql -v ON_ERROR_STOP=1 -1 -f migrate.sql` to wrap a whole script in a single transaction.

6 Import & Export Data

CSV in & out with \copy

```
-- Export a table
\copy users TO 'users.csv' CSV HEADER

-- Export a query result
\copy (SELECT id, email FROM users
      WHERE active) TO 'active.csv'
      CSV HEADER

-- Import (skips header)
\copy users FROM 'users.csv'
      CSV HEADER NULL ''
```

\copy vs COPY: \copy reads files on *your* machine. COPY uses *server* files and needs superuser perms.

Backup & restore (shell)

```
# dump a single database
pg_dump -Fc shop > shop.dump

# restore
pg_restore -d shop_new shop.dump

# dump + restore over a pipe
pg_dump shop | psql shop_clone

# schema only / data only
pg_dump --schema-only shop > ddl.sql
pg_dump --data-only shop > rows.sql
```

7 Diagnose & Tune

Health checks & lookups

```
-- who is connected right now?
SELECT pid, username, datname, state, query
FROM pg_stat_activity
WHERE state <> 'idle';

-- biggest tables
SELECT relname,
       pg_size_pretty(pg_total_relation_size(oid))
FROM pg_class
WHERE relkind = 'r'
ORDER BY pg_total_relation_size(oid) DESC
LIMIT 10;
```

```
-- ask the planner why
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM users WHERE email = 'a@b.io';

-- kill a runaway query
SELECT pg_cancel_backend(12345);    -- polite
SELECT pg_terminate_backend(12345); -- forceful

-- locks & blockers
SELECT * FROM pg_locks WHERE NOT granted;
```

Built-in help

\?	all psql meta-commands
\? variables	list of psql variables
\h	all SQL command names
\h SELECT	syntax for any keyword
\copyright	distribution terms

Fun & useful tricks

- ▶ Run \watch 1 after a slow query → live refresh.
- ▶ Make a ~/.psqlrc with \timing on and a custom PROMPT1.
- ▶ Press `Tab` `Tab` on an empty line for everything you can complete.
- ▶ \! ls runs a shell command without leaving psql.
- ▶ \dconfig shows server settings; \dconfig+ for sources.

Built with ♥ for PostgreSQL · keep this open in a tab next to your terminal · \? is your best friend